



1

21

3150 sp

ملاحق

كلية الهندسة المعلوماتية

السنة الثانية

10010110110101010101001001

BST

تعريف هامة:

الأشجار



- العرض

أكبر عدد للعقد في المستوى الواحد
- الحجم

عدد العقد الكلي في الشجرة
- المستوى

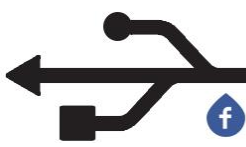
عدد طبقات الشجرة
- الارتفاع

أعلى ارتفاع لعقد في الشجرة (أي ارتفاع أبعد عُقدة عن الroot)
- الدرجة

هي أعلى درجة عقدة موجودة في الشجرة

العُقد

الأوراق	الحفيد	السلف	الدرجة	الارتفاع
<u>العقد الطرفية التي لا تتفرع منها أي عقد أخرى، أي أنها العقد التي لا تملك أبناء</u>	<u>الحفيد هو العقدة التي تقع على المستوى الثاني مباشرة تحت عقدة أخرى، أي أنه ابنُ لابن العقدة الأصل</u>	<u>أي عقدة تقع على المسار صعوداً من العقدة الحالية إلى الجذر</u>	<u>عدد أبناء هذه العقدة</u>	<u>عدد الوصلات من العقدة إلى أبعد ورقة تابعة لها.</u>

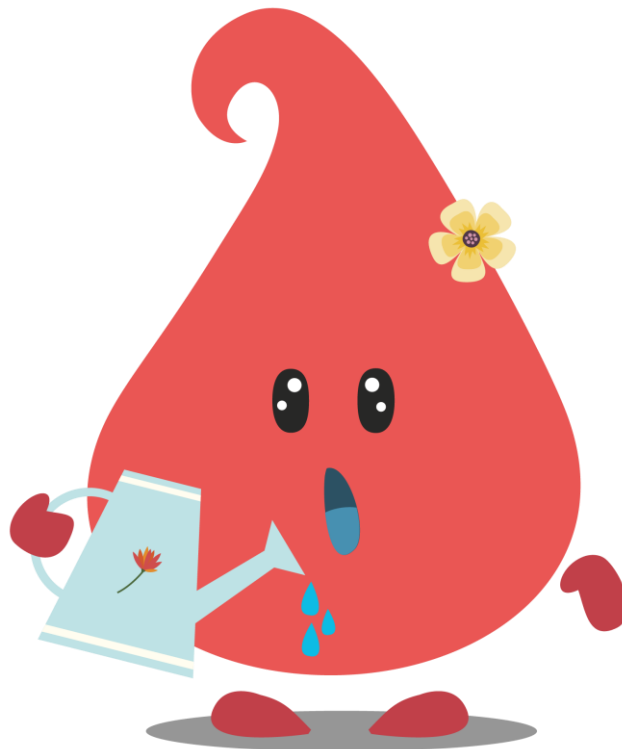




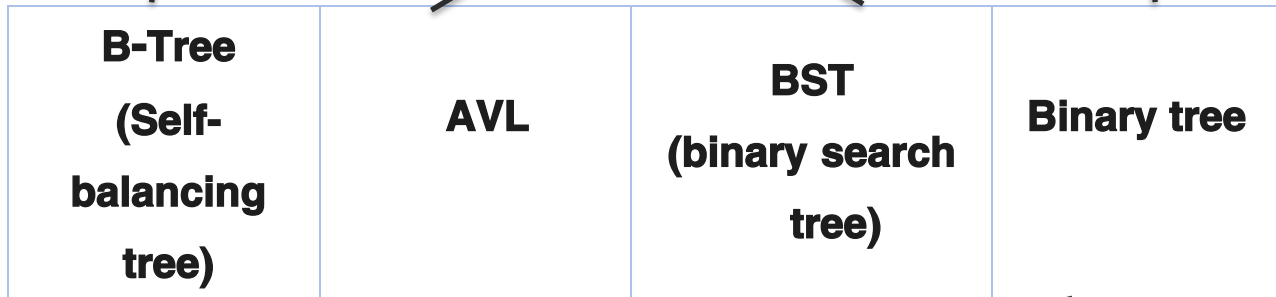
عقدة خارجية External node\leaf	عقدة داخلية internal node
تسمّى ورقة وهي عقدة ليس لها أبناء	أي عقدة لها ابن أو أكثر

ازرع جميلاً ولو في غير موضعه... فلا يضيع جميل أينما وُضِعَا
إنَّ الجَمِيلَ وإنْ طَالَ الزَّمانُ بِهِ فليسَ يحصدُهُ إلَّا الَّذي زَرَعَا..

بدل ما تزرع باينري تري، روو ازرع جميل (●'●)



أنواع الاشجار



Complete binary tree

Full binary tree

يجب ملء جميع المستويات بالكامل، باستثناء المستوى الأخير الذي يملأ من اليسار إلى اليمين.	كل عقدة تحتوي إما على 0 أو 2 من الأبناء فقط.
يمكن وجود عقدة بابن يساري واحد فقط (في نهاية الشجرة)	لا يمكن وجود عقدة بابن واحد (0 أو 2 فقط).
يجب ملء العقد من أقصى اليسار إلى اليمين بالتوالي. أي: تركز على "الكمال في الترتيب" (عدم ترك أي فراغ أثناء بناء الشجرة)	تركز على "تفرع" العقد (إما فرعين أو لا شيء).
تمثل بكفاءة عالية باستخدام المصفوفات (Arrays).	تمثل برمجياً عبر العقد المرتبطة. (Linked Nodes)
تمثيلها مثالي بالمصفوفات (Arrays): وذلك لأن العقد تشغل الخانات بالتوالي دون ترك أي فجوات، مما يوفر في الذاكرة ويجعل الوصول للأبناء سريعاً عبر المعادلات. $(2i, 2i+1)$	تمثيل مثالي بالعقد (Linked Nodes) يُفضل استخدام المؤشرات؛ لأن تمثيلها بالمصفوفات يترك فجوات خانات Null فارغة (وذلك للحفاظ على أماكن الأبناء المفقودين).

" تمارين شاملة " MCQs

ما هو القيد الأساسي في وصلات الشجرة؟

- A. تكون من الابن إلى الأب
- B. تكون من الأب إلى الابن فقط
- C. تكون ثنائية الاتجاه دائماً
- D. لا يوجد قيود

إذا كانت العقدة لها أبناء، فإنها تشكل ما يسمى بـ:

- A. شجرة كاملة
- B. شجرة جزئية (Sub-tree)
- C. مسار (Path)
- D. مستوى (Level)

الحل: B**الشرح:** العقدة مع أبنائها وأحفادها تشكل شجرة جزئية تنبثق من الشجرة الأم

إذا أردنا تخزين سلسلة نصية، فأي تصنيف سنختار بناءً على الرسم؟

- A. بنى معطيات خطية (Linear)
- B. بنى معطيات غير خطية (Non-Linear)
- C. بنى أساسية (Primitive)
- D. بنى معقدة

الحل: A

ما هو القيد البرمجي الصارم للاعتبار بنية المعطيات "شجرة"؟

- A. يمكن للعقدة أن تملك أكثر من أب.
- B. لكل عقدة أب واحد فقط، باستثناء الجذر.
- C. يجب أن يكون لكل عقدة ابنان دائماً
- D. الجذر هو العقدة التي لا تملك أبناء.

الحل: B

وفقاً للتعريف المعتمد في المحاضرة، ارتفاع العقدة هو:

- A. عدد الوصلات من العقدة إلى الجذر
- B. عدد الوصلات من العقدة إلى أبعد ورقة
- C. عدد العقد في مستواها
- D. عدد أشقاء العقدة

الحل: B

العقدة التي ليس لها أبناء تُسمى:

- A. عقدة داخلية. (Internal Node)
- B. ورقة. (Leaf/External Node)
- C. الجذر. (Root)
- D. السلف. (Ancestor)

الحل: B

عقدة تقع على المسار صعوداً من العقدة الحالية إلى الجذر تسمى:

- A. حفيد (Descendant)
- B. سلف (Ancestor)
- C. أخ (Sibling)
- D. ابن (Child)

الحل: B

تُصنف الشجرة بأنها "ثنائية" إذا كان:

- A. لكل عقدة ابنان بالضبط.
- B. لكل عقدة ابنان على الأكثر (0، 1، أو 2).
- C. ارتفاع الشجرة يساوي 2 دائماً.
- D. عدد عقدها الكلي زوجي

الحل: B

ما الفرق الجوهرى بين الشجرة الكاملة (Complete) والممتلئة (Full)؟

- A. الشجرة الممتلئة هي حالة خاصة من الكاملة.
- B. الشجرة الكاملة هي حالة خاصة من الممتلئة، ويشترط امتلاء العقد من اليسار لليمين
- C. الشجرة الكاملة لا يمكن أن تكون ثنائية.
- D. لا يوجد فرق، المصطلحان مترادفان

الحل: B

العدد الكلى للعقد في Full Binary Tree ارتفاعها H هو:

- A. 2^H
- B. $2^{(H+1)} - 1$
- C. $2 * H + 1$
- D. H^2

الحل: B

أعلى ارتفاع ممكن لشجرة ثنائية مكونة من n عقدة هو:

- A. $\log n$
- B. $n / 2$
- C. $n - 1$
- D. $\sqrt{n}$

الحل: C**الشرح:** يحدث الارتفاع الأقصى (n-1) عندما تكون الشجرة خطية (Linear)

عيب تمثيل الشجرة بالمؤشرات (Pointers) التقليدي هو:

- A. صعوبة الوصول للأبناء
- B. استهلاك ذاكرة قليل جداً
- C. لا يمكن معرفة العقدة الأب من الابن
- D. بطء القراءة

الحل: C**الشرح:** بسبب وجود مؤشر من الأب للابن فقط، لا يمكن الرجوع للأب ما لم يتم إضافة مؤشر ثالث.

عند تمثيل الشجرة الثنائية باستخدام مصفوفة أحادية، إذا كان الأب في الموقع i فأين يقع ابنه اليمين؟

A. $2i+1$

B. $2i$

C. $i+1$

D. $i/2$

الحل: A

الشرح: إذا كان الأب في الإندكس رقمها i فالابن اليساري بال $2i$ والابن اليميني بال $2i+1$

❖ إذا كان الجذر في الموقع 1 الابن الأيسر $2i$ والابن الأيمن $2i+1$

❖ إذا كان الجذر في الموقع 0 الابن الأيسر $2i+1$ والابن الأيمن $2i+2$

متى يعتبر تمثيل الشجرة بالمصفوفة الأحادية حلاً مثالياً وموفرًا للمساحة؟

A. عندما تكون الشجرة خطية

B. عندما تكون الشجرة كاملة (Complete)

C. عندما تكون الشجرة من الدرجة الثالثة.

D. عندما يكون عدد العقد فردياً

الحل: B

لحساب أصغر ارتفاع ممكن لشجرة ثنائية مكونة من n عقدة، نستخدم:

A. $n/2$

B. $n - 1$

C. $\log_2(n)$

D. 2^n

الحل: C

العلاقة الصحيحة لارتفاع الشجرة الثنائية H بدلالة عدد العقد n هي:

A. $\log_2(n) \leq H \leq n-1$

B. $n-1 \leq H \leq 2^n$

C. $1 \leq H \leq \log_2(n)$

D. $H=n$

في مصفوفة التمثيل الأحادي لشجرة ارتفاعها 3، ما هو حجم المصفوفة المطلوب حجزها

- A. 3 خانات
- B. 7 خانات
- C. 2 خانات
- D. 15 خانات

الحل: D قانون أقصى عدد للعقد $(h + 1)^2 - 1 = 16 - 1 = 15$

شجرة ثنائية 'مثالية' (Perfect Binary Tree) ارتفاعها 4. كم عدد العقد الكلي فيها؟

- A. 15
- B. 31
- C. 16
- D. 32

الحل: B

الشرح: بما أن $H=4$ ، فإن $2^{H+1} - 1 = 2^5 - 1 = 32 - 1 = 31$ عقدة

في التمثيل باستخدام مصفوفة أحادية يبدأ فهرسها من 1، إذا كانت العقدة في الموقع 7، فأين يقع 'الأب'؟

- A. 14
- B. 15
- C. 3
- D. 4

الحل: C

في خوارزمية التنقل من نوع Post_Order متى يتم تنفيذ أمر طباعة؟

- A. بعد زيارة الابن الأيسر وقبل زيارة الابن الأيمن
- B. بعد الانتهاء من زيارة كلا الابنين (الأيسر والأيمن)
- C. في بداية الدالة قبل أي استدعاء عودي
- D. null بمجرد العثور على عقدة

الحل: B

ترتيب الطباعة (للعقدة الحالية ال root)

Pre-order: الطباعة أولاً.

In-order: الطباعة في المنتصف.

Post-order: الطباعة آخرًا.

إذا قمنا باختيار شجرة بحث ثنائية (BST) بطريقة In-order، فالنتيجة هي:

A. أرقام عشوائية

B. أرقام مرتبة تنازلياً

C. أرقام مرتبة تصاعدياً

D. نفس ترتيب الإدخال

الحل: C

دائماً المرور بطريقة In-order :

(يسار -> جذر -> يمين) على شجرة يعطي عناصر مرتبة تصاعدياً من الصغير للكبير وهي ميزة شجرة البحث الثنائية (BST)

لإعادة بناء شجرة ثنائية بشكل فريد، نحتاج على الأقل إلى:

A. مسار واحد فقط

B. مساري Pre-order و Post-order

C. مسار In-order مع Pre-order أو Post-order

D. عدد العقد الكلي فقط

الحل: C

تُعرف الشجرة برمجيًا بأنها بنية:

A. تكرارية (Iterative).

B. عودية (Recursive).

C. خطية (Linear).

D. عشوائية (Random).

الحل: B

لأن كل عقدة فيها هي بحد ذاتها "جذر" لشجرة أصغر (Sub-tree)

هذا هو السبب في أن معظم خوارزميات الأشجار تكتب باستخدام الدوال العودية (Recursion)

إذا كان لدينا رسم (Graph) يحتوي على N من العقد و $N-1$ من الحواف، فما هو الشرط الإضافي ليصبح 'شجرة' حتماً؟

- A. أن يكون موجهاً (Directed)
- B. أن يكون متصلاً (Connected)
- C. أن يحتوي على جذر واحد فقط
- D. لا يحتاج لشروط إضافية

الحل : B

في شجرة ثنائية (Binary Tree)، إذا كان عدد الأوراق (Leaves) هو L ، فما هو عدد العقد التي لها ابنان بالضبط (Internal nodes with degree 2)؟

- A. L
- B. $L+1$
- C. $L-1$
- D. $2L$

الحل : C

قاعدة رياضية ثابتة في الأشجار الثنائية: عدد العقد ذات الابنين يساوي دائماً عدد الأوراق مطروحاً منه واحد

$$(N_2 = L - 1)$$

شجرة ثنائية ممتلئة (Full Binary Tree) تحتوي على 20 ورقة (Leaf). كم عدد العقد الداخلية (Internal Nodes) فيها؟

- A. 19
- B. 20
- C. 21
- D. 39

الحل : A

الشرح:

في الـ Full Binary Tree، كل عقدة داخلية لها ابنان. لذا نطبق القاعدة:

$$\text{عدد العقد الداخلية} = \text{عدد الأوراق} - 1 = 20 - 1 = 19$$

كم عدد الوصلات (null pointers) في شجرة ثنائية تحتوي على n عقدة إذا مثلناها باستخدام المؤشرات؟

- A. N
- B. $N+1$
- C. $N-1$
- D. $2N$

الحل : B

المحاضرة الثانية عملي:

ما هو خرج الكود التالي إذا كانت الشجرة فارغة (root == null) ؟

```
System.out.println(ifNodeExists(root, 10));
```

- A. True
- B. False
- C. 0
- D. Null

الحل : B

في دالة buildTree ، لماذا نستخدم $inIndex - 1$ عند بناء الشجرة اليسرى؟

- A. لتقليل استهلاك الذاكرة
- B. لتحديد نهاية نطاق العناصر التي تقع على يسار الجذر في مصفوفة inorder
- C. لأن الشجرة دائماً تكون مائلة لليسر
- D. للانتقال للعنصر السابق في مصفوفة preorder

الحل : B

ابتسامتك كالحلوى تمنح اليوم نكهة خاصة 😊

ما هي الحالة القاعدية (Base Case) التي تضمن توقّف البحث في دالة ifNodeExists إذا لم يتم العثور على القيمة؟

- A. if (node.data == key) return false;
- B. if (node == null) return false;
- C. if (node.left == null) return true;
- D. return ifNodeExists(node.right, key);

الحلّ: B

التفسير: عندما تصل الدالة إلى نهاية الغصن (null) دون العثور على المفتاح، تعيد false للدلالة على عدم الوجود

ما هو مخرج كود getLeafCount لشجرة فارغة؟

```
int getLeafCount(Node node) {  
    if (node == null) { return 0; }  
    if (node.left == null && node.right == null) { return 1; }  
    else { return getLeafCount(node.left) + getLeafCount(node.right); }
```

- A. 1
- B. 1-
- C. 0
- D. NullPointerException

الحلّ: C

أي تعليمة تُستخدم لربط الابن الأيسر بالعقدة الحالية في خوارزمية البناء؟

- A. tNode.left = buildTree(in, pre, inStrt, inIndex - 1);
- B. tNode.left = pre[preIndex++];
- C. tNode.left = buildTree(in, pre, inIndex - 1, inEnd);
- D. tNode.right = tNode.left;

الحلّ: A

لماذا نستخدم $inStrt > inEnd$ كشرط توقف في بناء الشجرة؟

- A. للإشارة إلى أن المصفوفة فارغة أو الجزء المراد بناؤه انتهى.
- B. لأن Preorder انتهى.
- C. لمنع تكرار العقد.
- D. لحساب الارتفاع.

الحل: A

التفسير: إذا تجاوزت البداية النهاية، فهذا يعني لا توجد عناصر في هذا النطاق لبنائها.

إذا كان لدينا Inorder: B A C و Preorder: A B C؛ فما هو الابن الأيسر لـ A؟

- A. C
- B. B
- C. لا يوجد
- D. نفسها A

الحل: B

```
int countNonleaf(Node root) {
    if (root == null || (root.left == null && root.right == null)) return 0;
    return 1 + countNonleaf(root.left) + countNonleaf(root.right); }
```

لماذا نستخدم + 1 في السطر الأخير من الدالة؟

- A. وليست ورقة null لإضافة العقدة الحالية للحساب لأننا تأكدنا أنها ليست
- B. لأن الشجرة دائماً تبدأ من الرقم 1.
- C. لتعويض العقدة التي يتم حذفها أثناء البحث.
- D. لزيادة ارتفاع الشجرة بمقدار واحد.

الحل: A

اعتبرها محاضرة وجبتك الشهية فلا تتركها تبرد 🍗🍟😊

```
static int preIndex = 0;
```

```
Node buildTree(char in[], char pre[], int inStrt, int inEnd) {
```

```
Node tNode = new Node(pre[preIndex++]);
```

```
}
```

ما الذي سيحدث لو قمنا بتعريف static int preIndex = 0; داخل الدالة (Local variable) بدلاً من تعريفها كـ خارجها؟

A. سيعمل الكود بشكل أسرع

B. سيتم بناء الشجرة بشكل صحيح ولكن بترتيب عكسي.

C. سيفشل بناء الشجرة لأن كل استدعاء عودي سيعيد ضبط المؤشر إلى 0، مما يؤدي لتكرار الجذر الأول فقط.

D. لن يترجم الكود (Compilation Error)

الحل: C

النظري:

Binary search tree :

هي شجرة ثنائية شرطها أن تكون جميع القيم الموجودة في الفرع اليسار للعقدة أصغر من قيمة العقدة وجميع القيم الموجودة في الفرع الأيمن للعقدة أكبر من قيمة العقدة

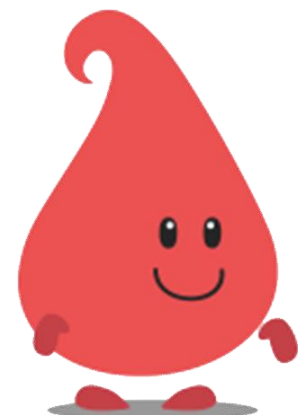
الإضافة في BST

إضافة عُقدة كَجَذَر

1. نقوم بتفكيك الشجرة وإعادة تشكيلها من البداية، أي نضع الرقم الذي نريد إضافته كَجَذَر ونضيف باقي العناصر بعده عُنْصراً تلو الآخر وفق مبدأ BST
 2. نقارن قيمة العقدة المراد إضافتها كَجَذَر جديد للشجرة مع قيمة الجذر السابق:
 - . إذا كانت أصغر منه نقل الجذر السابق مع شجرته الفرعية لتصبح الابن الأيسر للجذر الجديد
 - . وإلا نقل الجذر السابق مع شجرته الفرعية لتصبح الابن الأيمن للجذر الجديد
- وهكذا تمكّنّا من إضافة جذر جديد دون الحاجة للمرور على جميع العناصر ومقارنتها بالجذر الجديد

إضافة عُقدة كَوَرقَة

نختار المكان الصحيح كورقة بحيث لا تكسر شرط ال BST



Deletion in BST

للعقدة ابنان:

نستبدل العقدة التي نريد حذفها
بالعنصر التالي successor ثم نحذفه
من مكانه القديم لمنع التكرار
التعقيد $O(h)$

للعقدة ابن وحيد:

نحذف العقدة ثم نجعل
مؤشر أبيها يُشير إلى ابنها
الوحيد
التعقيد $O(1)$

حذف عقدة ليس لها أبناء

(حذف leaf):
نجعل الأب للورقة المراد
حذفها يُشير إلى قيمة null
التعقيد $O(1)$

العُنصر التَّالي successor

وهو العنصر الذي يلي العقدة المطلوبة
مباشرةً من حيث القيمة

ليس للعقدة ابن يمين:

العنصر التالي هو أول أب أكبر من
هذه العقدة

للعقدة ابن يمين:

يكون successor العقدة التي تقع أقصى اليسار
في الشجرة الفرعية اليمينية لهذه العقدة
أي أصغر قيمة في القسم اليمين

ما هو الشرط الأساسي لشجرة البحث الثنائية (BST)؟

- A. كل عقدة لها فرعين بالضبط دائماً
- B. القيم في الأيسر أصغر من العقدة، والقيم في الأيمن أكبر منها
- C. القيم في الأيمن أصغر من العقدة، والقيم في الأيسر أكبر منها
- D. الشجرة يجب أن تكون متوازنة دائماً

الجواب: B

ما هو التعقيد الزمني للبحث في شجرة BST خطية (linear/skewed)؟

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$

$O(n^2)$.D**الجواب: C**الحالة الأسوأ عندما تكون الشجرة خطية هي $O(n)$

ما هو تعقيد إضافة عقدة كجذر باستخدام الطريقة الأولى (التفكيك وإعادة)؟

A. $O(\log n)$ B. $O(n)$ C. $O(n \times H)$ D. $O(H)$ **الجواب: C**

لإضافة جذر، إذا كانت القيمة المضافة أصغر من الجذر القديم، فإن الجذر القديم يصبح:

A. ابن أيسر للجذر الجديد

B. ابن أيمن للجذر الجديد

C. يتم حذفه

D. يبقى في مكانه

الجواب: B

ما فائدة إضافة العقد الجديدة كجذر (Root Add)؟

A. دائماً تجعل الشجرة متوازنة

B. تسرع البحث عن العناصر المضافة مؤخراً

C. تقلل الارتفاع دائماً

D. تمنع تكرار القيم

الجواب: B**الشرح:** مثل محرك البحث اذا كنا نبحث عن شيء دائماً يظهر أول شيء

كيف نجد ال Successor لعقدة تمتلك ابن يميني؟

A. أقصى اليمين في الشجرة الفرعية اليسارية

B. أقصى اليسار في الشجرة الفرعية اليمينية

C. أول أب أكبر منها

D. الابن الأيسر للابن الأيمن دائماً

الجواب: B

ما هو تعقيد إيجاد العنصر التالي؟

A. $O(1)$

B. $O(n)$

C. $O(\log n)$

D. $O(H)$

الجواب: D

ما هو الـ Predecessor للعقدة؟

A. أصغر عنصر في الشجرة

B. أكبر عنصر في الفرع الأيسر للعقدة

C. أصغر عنصر في الفرع الأيمن للعقدة

D. الأب المباشر

الجواب: B

أين نجد 'أكبر عنصر' في شجرة BST؟

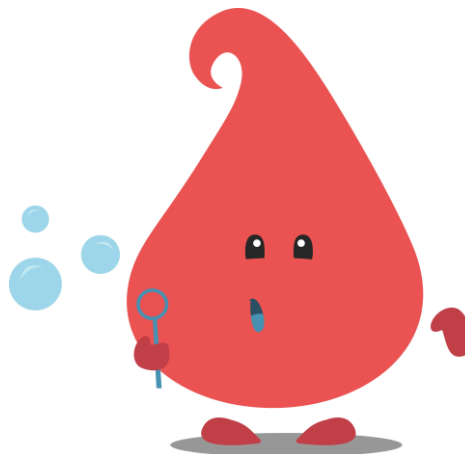
A. في الجذر

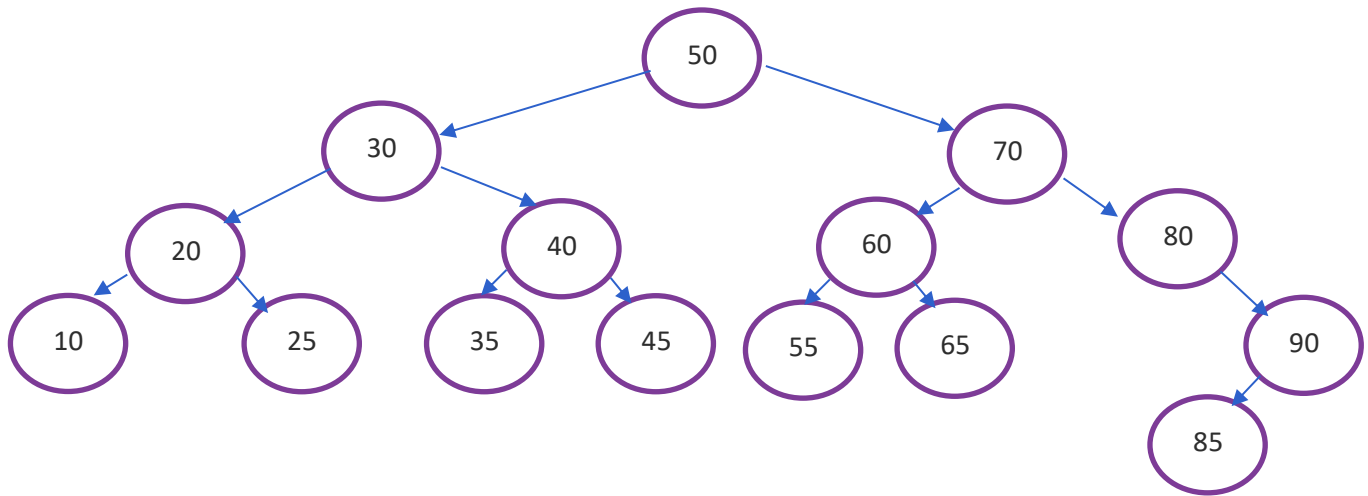
B. في أقصى اليمين

C. في أقصى اليسار

D. في أي ورقة

الجواب: B





بالنظر للشجرة المرجعية، ما هو الـ Successor (العنصر التالي) للعقدة 45؟

- A. 50
- B. 40
- C. 35
- D. لا يوجد

الجواب: A

الشرح: العقدة 45 ليس لها ابن يميني، لذا نصعد للأعلى حتى نجد أول أب أكبر منها، وهو الجذر 50.

ما هو الـ Successor للعقدة 50 (الجذر) في هذه الشجرة؟

- A. 70
- B. 60
- C. 55
- D. 51

الجواب: C

للجذر ابن يميني (70)، لذا الـ Successor هو أصغر عنصر في شجرته اليمينية (أقصى يسار الـ 70)، وهو 55.

إذا قمنا بحذف العقدة 30، أي عنصر سيحل محلها (Successor) لإعادة التوازن؟

- A. 20

B. 40

C. 35

D. 25

الجواب: C

العقدة 30 لها ابنان، نستبدلها بالـ Successor الخاص بها وهو 35 (أصغر عنصر في شجرتها اليمينية).

ما هو ارتفاع (Height) هذه الشجرة المرجعية؟ (باعتبار الجذر في المستوى 0)

A. 3

B. 4

C. 5

D. 6

الجواب: B

أعمق مسار هو $50 \leftarrow 70 \leftarrow 80 \leftarrow 90 \leftarrow 85$ ، مما يعطي 4 مستويات بعد الجذر.

أين سيتم وضع العنصر 32 إذا قررنا إضافته للشجرة؟

A. يسار 30

B. يمين 35

C. يسار 35

D. يمين 40

الجواب: C**الشرح:**

$50 > 32$ (يسار)، $30 < 32$ (يمين)، $40 > 32$ (يسار)، $35 > 32$ (يسار).

عند حذف العقدة 80، ما هو الإجراء المتبع؟

A. استبدالها بـ 90 مباشرة

B. استبدالها بـ 85

C. جعل الـ 70 مؤشر مباشرة للـ 90

D. لا يمكن حذفها لأنها ليست ورقة

الجواب: C

العقدة 80 لها ابن وحيد (90)، لذا نحذفها ونجعل الأب (70) يشير مباشرة لابنها (90).

أي من العقد التالية ليس لها Successor في هذه الشجرة؟

A. 85

B. 90

C. 80

D. 45

الجواب: B

الشرح : لأن 90 أكبر عقدة بالشجرة لا يوجد عقدة أكبر منها

ما هو تعقيد عملية البحث عن العقدة 85 في هذه الشجرة تحديداً؟

A. $O(1)$

B. 4 مقارنات

C. 5 مقارنات

D. $O(n)$

الجواب: C

المسار هو: 50 ثم 70 ثم 80 ثم 90 ثم 85. (5 مقارنات حتى الوصول للعقدة).

اين نجد أصغر عنصر في هذه الشجرة؟

A. أقصى اليمين

B. أقصى اليسار

C. الجذر

D. تحت الـ 80

الجواب: B

دائماً أصغر عنصر في الـ BST هو في أقصى اليسار، وهو هنا 10.

عند حذف العقدة 20، ماذا يحدث للعقدة 25؟

A. تصبح ابناً أيسر لـ 30

B. تصبح ابناً أيسر لـ 50

C. تصبح ابناً أيمن لـ 10

D. تختفي



الجواب: A العقدة 20 لها ابنان، يتم استبدالها بـ 25 (ال Successor الخاص بها).

ما هو الـ Predecessor للعقدة 30؟

A. 25

B. 20

C. 10

D. 35

الجواب: A

أكبر عنصر في الشجرة اليسارية لـ 30 هو 25.

وجبة الخوارزميات كانت دسمة جداً لا تنسوا كاسة الشاي

نلتاقكم في الملحق القادم 😊

